



TBWB

Angular Advanced White Label / Multi tenant-solutions



Peter Kassenaar
info@kassenaar.com



White Label / Multi tenant-solutions

Single core app; various routes, features, services, UI, etc per client.

Whitelabel / multi tenant solutions

- Desire:
 - Having one (1) "Core" application with basic functionality, layout, navigation etc.
 - Specific clients can have specific Features, Routes, UI, Logic, Service, etc, based on their specific needs.
 - Every client has its own "feature pack".

- Question:

"What would be the best way to organize such a solution?"

Typical Separation of this architecture

- Core application:
 - **Shell**/layout, skeleton app, authentication, http interceptors, design system (preferably in themes), generic features
- Customer Features:
 - Every client has its **own package of features** and additions
 - Extra pages, extra routes, other feature flags, different config layouts, other branding/theme, etc.
- **Important!**
 - “Variation” here is extra config, or plug-in modules.
 - NOT `if customer === 'XYZ'` statements in the code.

Dynamic loading – options per customer

- Goal: “keep *initial bundle small* AND do not ship irrelevant code to customers”.
- Basically three (3) levels available:
 1. Build-time selection (simple, but multiple builds)
 2. Runtime selection within a single build (most common)
 3. Truly dynamic plugins/microfrontends (heavy, but super flexible)

Roughly three options for white labeling

1. **Build-time selection** (simple, but multiple builds)
 1. file replacements / env / customer-specific builds
 2. This works, but you lose the '*build once, deploy everywhere*' advantage
2. **Runtime selection** within a single build (most common)
 1. runtime config determines which routes/features are 'on'
 2. lazy loading via router (`loadChildren` / `loadComponent`)
 3. possibly *guards* (`canMatch`) to make routes conditionally matchable (useful for feature flags)
3. **Truly dynamic plugins/microfrontends** (heavy, but super flexible)
 1. Module Federation / Native Federation: customer plugins can even be deployed separately
 2. useful if customers have completely different feature sets or teams deploy separately

Some background info: route guards

The screenshot shows the Angular documentation page for 'Control route access with guards'. On the left is a sidebar with a 'Routing' section containing a list of topics: Overview, Define routes, Show routes with Outlets, Navigate to routes, Read route state, Redirecting routes, Control route access with guards (highlighted in purple), Route data resolvers, Lifecycle and events, Testing routing and navigation (marked 'New'), Other routing tasks, Creating custom route matches, and Rendering strategies (marked 'New'). The main content area has a breadcrumb 'In-depth Guides > Routing' and a title 'Control route access with guards' with an edit icon. Below the title is a critical warning box with a red triangle icon: 'CRITICAL: Never rely on client-side guards as the sole source of access control. All JavaScript that runs in a web browser can be modified by the user running the browser. Always enforce user authorization server-side, in addition to any client-side guards.' This is followed by a paragraph explaining that route guards are functions that control navigation and are like checkpoints for user access. Below this is a section titled 'Creating a route guard' which states that route guards can be generated using the Angular CLI. A code block shows the command: '\$ ng generate guard CUSTOM_NAME', with a copy icon to its right.

← Routing Updated

Overview

Define routes

Show routes with Outlets

Navigate to routes

Read route state

Redirecting routes

Control route access with guards

Route data resolvers

Lifecycle and events

Testing routing and navigation New

Other routing tasks

Creating custom route matches

Rendering strategies New

In-depth Guides > Routing

Control route access with guards

⚠ CRITICAL: Never rely on client-side guards as the sole source of access control. All JavaScript that runs in a web browser can be modified by the user running the browser. Always enforce user authorization server-side, in addition to any client-side guards.

Route guards are functions that control whether a user can navigate to or leave a particular route. They are like checkpoints that manage whether a user can access specific routes. Common examples of using route guards include authentication and access control.

Creating a route guard

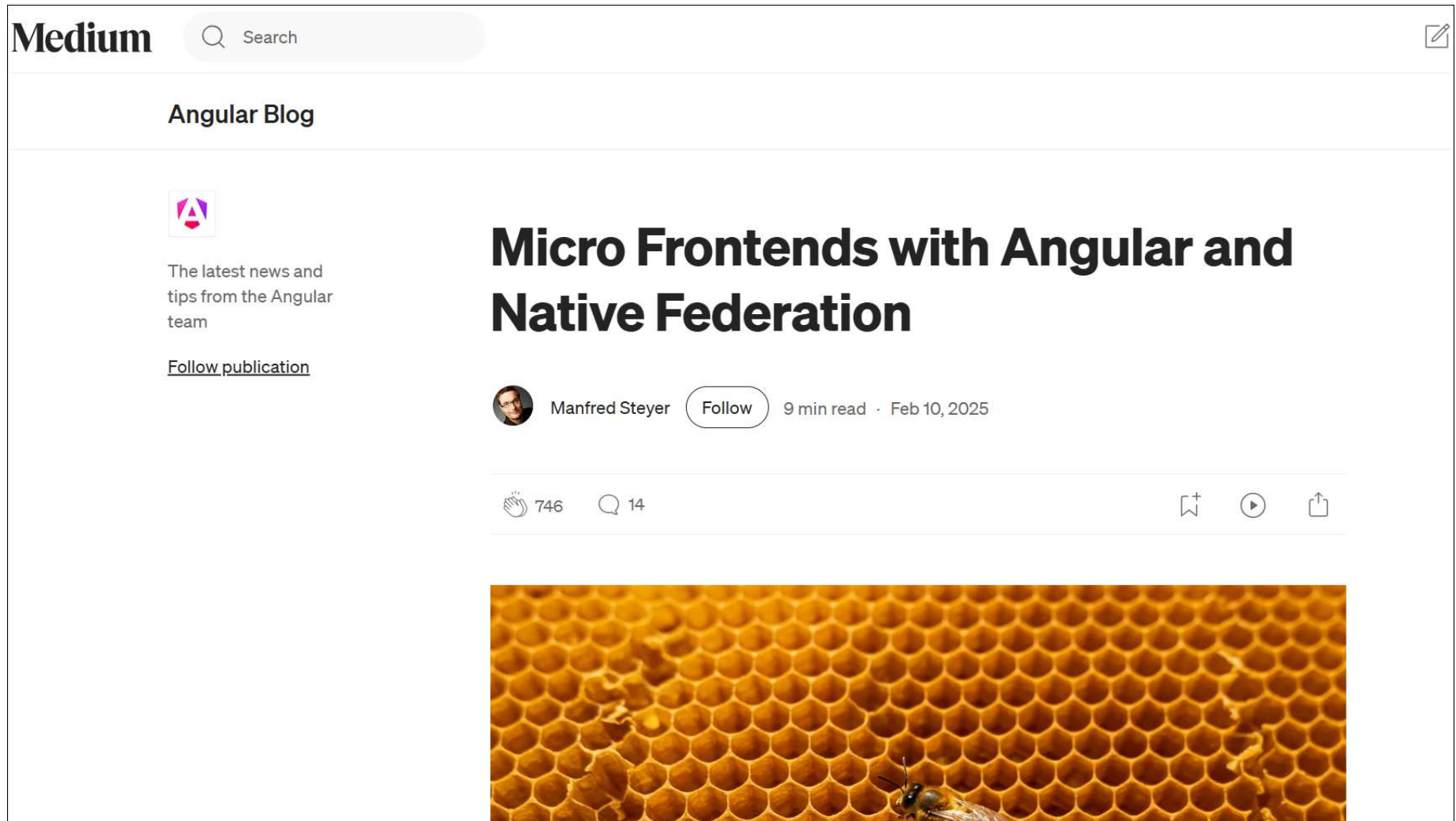
You can generate a route guard using the Angular CLI:

```
$ ng generate guard CUSTOM_NAME
```

<https://angular.dev/guide/routing/route-guards>

Micro frontends and Module federation

(warning: heavy stuff!)



<https://blog.angular.dev/micro-frontends-with-angular-and-native-federation-7623cfc5f413>

Example: 'feature packs' per customer

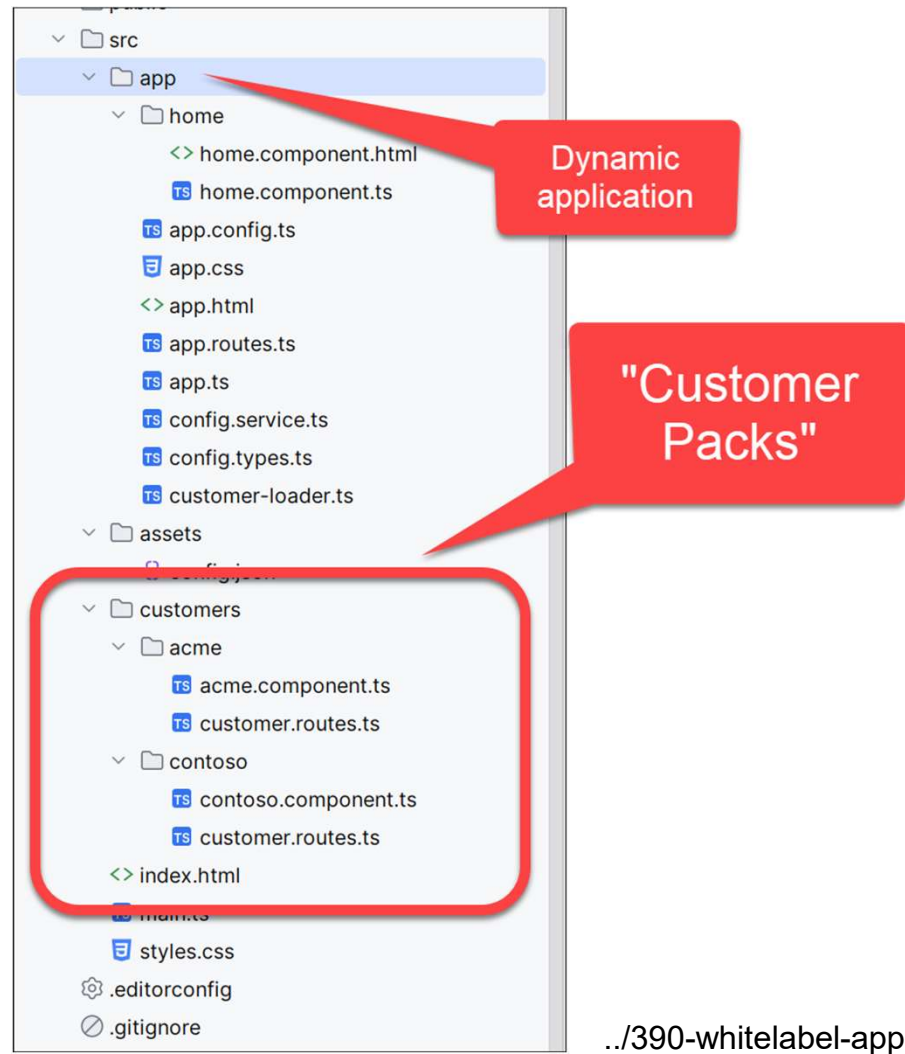
- Example solution:

*Every customer has one (1) lazy-loaded route bundle (acting as a "**customer feature pack**") which is only loaded once the customer is active.*

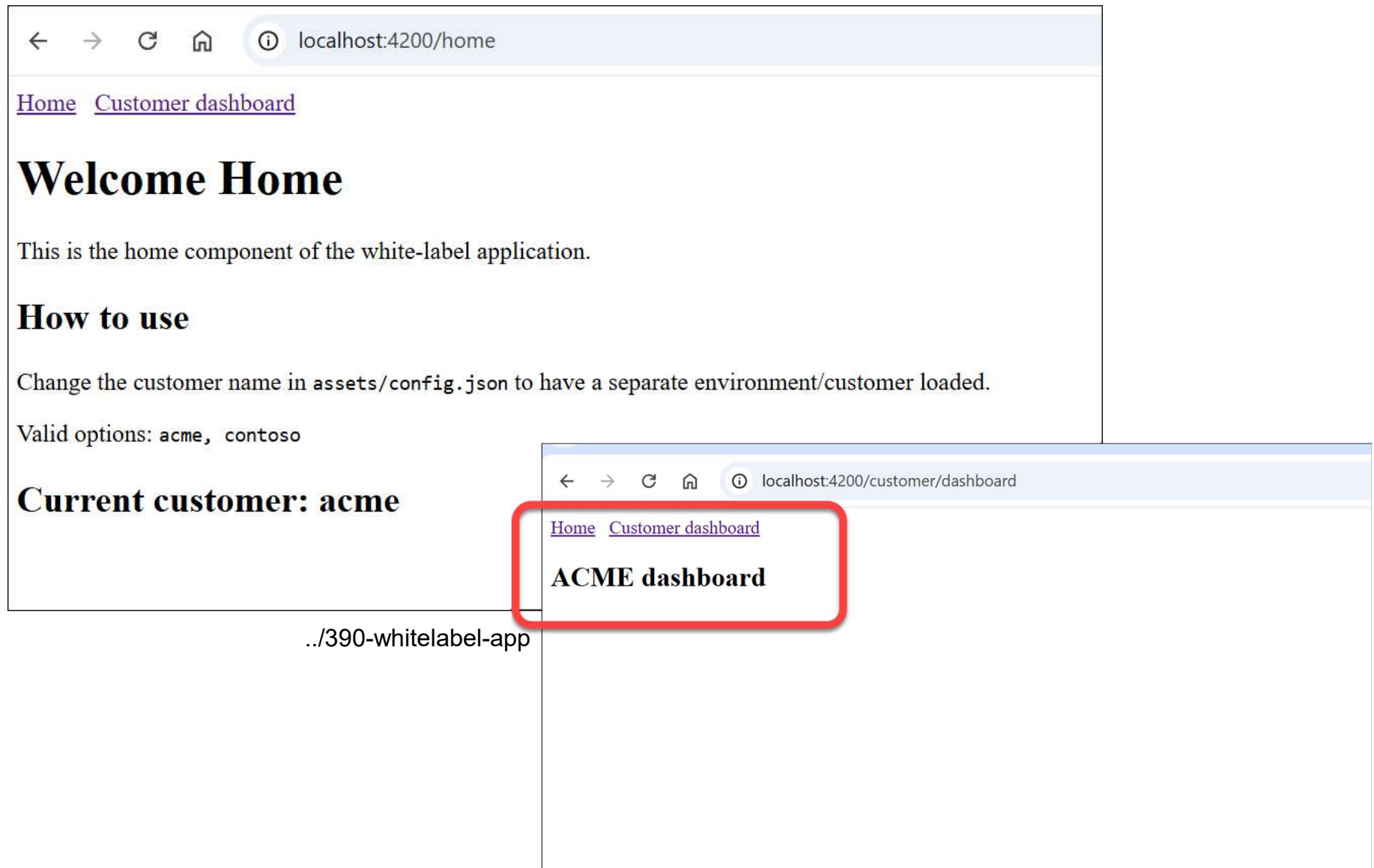
This means in code...

- There is **one core app**
- There are multiple **customer folders**
- each customer has **their own** `routes.ts` export
- selection occurs at **runtime** (via config, like in dynamic applications)
- **Lazy loading** (via `import()`) in main `app.routes.ts`.

App Structure / architecture



When customer `acme` is active:



../390-whitelabel-app

assets/config.json + config.types.ts

- Decide **which customer** + **which features** are being loaded:

```
{  
  "customer": "acme",  
  "validCustomers": ["acme", "contoso"],  
  "features": {  
    "apiUrl": "https://api.test.com",  
    "featureX": true,  
    "featureY": false  
  }  
}
```

```
//config.types.ts  
export interface AppRuntimeConfig {  
  customer: string;  
  validCustomers: string[];  
  features: {  
    apiUrl: string;  
    featureX: boolean;  
    featureY: boolean;  
  };  
}
```

config.service.ts (see also “dynamic applications”)

```
//config.service.ts
import { Injectable } from '@angular/core';
...

@Injectable({ providedIn: 'root' })
export class ConfigService {
  private _config!: AppRuntimeConfig;

  get config(): AppRuntimeConfig {
    if (!this._config) throw new Error('Config not loaded yet');
    return this._config;
  }

  constructor(private http: HttpClient) {}

  async load(): Promise<void> {
    const url = `assets/config.json?v=${Date.now()}`;
    this._config = await firstValueFrom(this.http.get<AppRuntimeConfig>(url));
  }
}
```

customer.loader.ts

(providing a map of customer routes, called by `provideRouter`)

```
//customer-loader.ts
import {Routes} from '@angular/router';

export type CustomerKey = 'acme' | 'contoso';

const CUSTOMER_ROUTES: Record<CustomerKey, () => Promise<Routes>> = {
  acme: () => import('../customers/acme/customer.routes')
    .then(m => m.CUSTOMER_ROUTES),
  contoso: () => import('../customers/contoso/customer.routes')
    .then(m => m.CUSTOMER_ROUTES),
};

export function loadCustomerRoutes(customer: CustomerKey): Promise<Routes> {
  return CUSTOMER_ROUTES[customer]();
}
```

app.routes.ts (basic, core routes)

```
// app.routes.ts
import { Routes } from '@angular/router';

export const CORE_ROUTES: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'home' },
  { path: 'home', loadComponent: () =>
import('./home/home.component').then(m => m.HomeComponent) },

  // placeholder: is being replaced during init!
  { path: 'customer', children: [] },

  { path: '**', redirectTo: 'home' },
];
```


app.config.ts

```
// app.config.ts
import { loadCustomerRoutes } from './customer-loader';
...
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideRouter(CORE_ROUTES), // using the CORE_ROUTES placeholder

    provideAppInitializer(async () => {
      ...
      const customerRoutes = await loadCustomerRoutes(customer);

      // The magic happens here
      const next = CORE_ROUTES.map(r =>
        r.path === 'customer' ? { ...r, children: customerRoutes } : r
      );

      // reset router
      router.resetConfig(next);
    }),
  ],
};
```

Routes per customer:

```
// customer.routes.ts :: customer ACME  
import { Routes } from '@angular/router';  
import { AcmeComponent } from './acme.component';  
  
export const CUSTOMER_ROUTES: Routes = [  
  { path: 'dashboard', component: AcmeComponent },  
];
```

```
// customer.routes.ts :: customer CONTOSO  
import { Routes } from '@angular/router';  
import { ContosoComponent } from './contoso.component';  
  
export const CUSTOMER_ROUTES: Routes = [  
  { path: 'dashboard', component: ContosoComponent },  
];
```

Components + other stuff per customer

```
// AcmeComponent
import { Component } from '@angular/core';

@Component({
  standalone: true,
  template: `<h2>ACME dashboard</h2>`,
})
export class AcmeComponent {}
```

```
// ContosoComponent
import { Component } from '@angular/core';

@Component({
  standalone: true,
  template: `<h2>CONTOSO dashboard</h2>`,
})
export class ContosoComponent {}
```

This example is NOT:

- Microfrontends
- Module federation
- Multiple builds
- runtime eval / dynamic path string
- if/else customer checks in components

Signals when to look further

Signal	Next steps
Multiple teams per customer	Module Federation
Independent deploys	Microfrontends
Third Party Plugins	Remote MFEs
Runtime URLs per tenant	Native Federation

Questions

- Q: Do I distribute / build **all customer artifacts**?
- A: Yes, You distribute **one build artefact** (the *dist/* output). That build does contain code for all customer packs (as separate lazy chunks). But runtime config determines which chunk(s) are ever retrieved.
- **Often: distribute one /build or Docker image, with specific config.json's**

Per customer separate location?

- **Publishing** is twice, but **building** is once!
 - `https://acme.example.com/` → same build, own `config.json`
 - `https://contoso.example.com/` → same build, own `config.json`

More concerns

- Q: "But then I ship **customer code to everyone...** is that okay?"
- A: "**That's the trade-off with this option**"
 - Pro: 1 build, simple, little infrastructure
 - Con: you 'ship' all customer code (although it's only downloaded when you navigate to it)
- In many B2B SaaS scenarios, **that's fine**, because it's **client-side code anyway** (no secrets)
 - the extra bytes are mainly in chunks that are never loaded. You save a lot of build/deploy complexity
- When is it NOT okay?
 - If customers are absolutely not allowed to see each other's UI/features (compliance/contractual)
 - if bundles become too large
 - you really want to deploy different teams/versions per customer to: